

EVNet Sentinel: A Leakage-Corrected Reproduction and Comparative Study of Intrusion Detection on Electric Vehicle Charging Networks

Mukta Varak
Dept. of Computer Engineering
Vidyalankar Institute of Technology
Mumbai, India
24102A0013

Shruti Chaurasiya
Dept. of Computer Engineering
Vidyalankar Institute of Technology
Mumbai, India
24102A0018

Shardul Chogale
Dept. of Computer Engineering
Vidyalankar Institute of Technology
Mumbai, India
24102A0021

Neha Chavhan
Dept. of Computer Engineering
Vidyalankar Institute of Technology
Mumbai, India
24102A0022

Abstract—Electric vehicle charging stations are networked computers whose compromise can halt charging, metering and billing. We present EVNet Sentinel, a network intrusion detection system for the link between charging stations and their management system, and use it to reproduce and audit the online-learning detector of Makhmudov et al. on the CICEVSE2024 dataset. We find that the published preprocessing retains absolute capture timestamps: a decision tree given one timestamp column alone recovers the fifteen-class label with 99.76 percent accuracy, above the published 98.40 percent. We further show that the drift detector fires on the seams between concatenated capture files, and that the high accuracy of streaming evaluation in capture order is matched by simply repeating the previous label. With the leakage removed and every result averaged over three seeds, volumetric floods remain perfectly separable while the six nmap reconnaissance modes are not, and an information-theoretic analysis shows this ceiling is a property of the flow features rather than of model capacity. A per-attack test harness finds that no attack passes as normal traffic, that scans are recognised as reconnaissance but rarely named exactly, and that Random Forest offers the best naming with the fewest false alarms. Requiring confidence before naming an attack converts most wrong scan names into correct category alerts. The system is delivered with a prediction service, role-based response guidance and an interactive front end built on real held-out flows.

Index Terms—Intrusion detection, electric vehicle charging infrastructure, CICEVSE2024, data leakage, adaptive random forest, ADWIN, concept drift, OCPP

I. INTRODUCTION

The transition to electric mobility has produced a large, interconnected cyber-physical infrastructure of Electric Vehicle Charging Stations (EVCS). A public charger is a networked computer that communicates over two standard protocols: ISO 15118 between the vehicle and the station (Electric Vehicle Supply Equipment, EVSE), and the Open Charge Point Protocol (OCPP) between the station and its Charging Station Management System (CSMS). The OCPP link carries

authorisation, meter values and remote control, so disrupting it stops drivers from charging and operators from billing. These links expose charging networks to denial of service (DoS), reconnaissance, false-data injection and host-level compromise [1], [3].

Machine-learning intrusion detection systems (IDS) trained offline cannot adapt when traffic changes, a phenomenon known as *concept drift*. Makhmudov et al. [4] addressed this with the first online, streaming IDS for charging systems: an Adaptive Random Forest (ARF) [6] combined with Adaptive Windowing (ADWIN) drift detection [7], reporting 99.13% binary and 98.40% multiclass accuracy on the CICEVSE2024 dataset [8].

This paper reproduces that work and asks whether those figures measure the detection of attacks or a property of how the dataset was assembled. We present **EVNet Sentinel**, a network IDS for the station-to-CSMS link, and use it as the vehicle for the audit. Our contributions are:

- 1) **Leakage audit.** Independently verifiable findings: timestamp leakage, a discarded legitimate feature, label autocorrelation in stream order, drift alarms that coincide with capture-file seams, and a residual port fingerprint (Section V).
- 2) **Corrected benchmarks** of four static classifiers and ARF+ADWIN, reported as mean and standard deviation over three seeds with macro- F_1 as the headline metric (Section VIII).
- 3) **An explanation of the reconnaissance ceiling** from the structure of the errors and a per-feature mutual-information analysis (Section VIII-C).
- 4) **Per-attack evaluation and model choice:** detection, categorisation and naming rates per attack and model, false-alarm rates, and a confidence-gated naming rule (Sections VIII-D–VIII-E).

- 5) **A provenance analysis of the testbed** showing that every attack targets a charging station and that the data were recorded passively (Section IV-B).
- 6) **A working system:** prediction service, role-based response guidance and an interactive front end (Section VI).

Section II reviews related work, Section III formalises the task, Section IV describes the data, Sections V–VII present the audit, system and method, Section VIII reports results, and Sections IX–XI cover reproducibility, limitations and conclusions.

II. LITERATURE REVIEW

This section reviews the five works that shaped EVNet Sentinel, states the gap each leaves, and compares them.

A. Bean et al. (2026): EVSE cybersecurity survey [1]

Bean et al. survey ML-based intrusion detection for EVSE across network, host and power-consumption monitoring. They formalise the IDS lifecycle as traffic capture, labelling, modelling, deployment and forensics, and review 24 studies on CICEVSE2024. They find that CICEVSE2024 is the dominant benchmark, that Random Forest and ensembles are strongest on flow data, and that concept drift, scalability and application-layer (OCPP message) monitoring remain open.

Gap: the survey takes reported accuracies at face value and does not audit the preprocessing behind them.

B. Joglekar et al. (2026): hybrid NIDS and HIDS [2]

Joglekar et al. combine a network IDS with a host IDS. Their network component, XGBoost on 547,834 CICEVSE2024 flows with 67 features, reaches 99.99% accuracy; the host component reaches 96.60% on host events and 70.34% on power traces, and the hybrid 83.47%. They remove IP, MAC and vendor identifiers to prevent leakage and show that flow-level analysis outperforms packet level.

Gap: timestamps are retained, so the 99.99% is exposed to the capture-session fingerprint of Section V; training is offline and cannot adapt to drift.

C. Malimage et al. (2023): EV cybersecurity challenges [3]

Malimage et al. analyse the problem from an industry and regulatory perspective: real breaches involving vehicle and charging platforms, the attack surface across grid, communication and user domains, weaknesses of OCPP and OCPI, and the relevant frameworks (NIST, ISO/SAE 21434, UN R155). They note that routing many charge points through one location creates a single point of failure with grid-level consequences.

Gap: the paper motivates detection but proposes no detector.

D. Makhmudov et al. (2025): online ML for EVCS IDS [4]

This is the baseline we reproduce. ARF with 20 trees, a 50% feature subset per split and ADWIN ($\delta = 0.002$) is run as a streaming learner on the CICEVSE2024 network traffic, reporting 99.13% binary and 98.40% multiclass accuracy, 12 and 11 drift events, and 3.7 ms per instance. Drift handling is presented as the method’s key advantage.

Gap: the preprocessing retains six absolute timestamp columns and discards `dst_port`; evaluation in capture order is inflated by label autocorrelation; and the detected drift coincides with file seams (Section V).

E. Tanyildiz et al. (2025): RUL-GAN pre-warning [5]

Tanyildiz et al. predict *when* the next attack will occur rather than whether a flow is malicious. A GAN generates synthetic attack samples for LSTM, GRU, RNN, CNN and MLP regressors of the time to the next attack; the best (GAN-MLP) reaches $R^2 = 0.7379$ on the EVS One dataset.

Gap: a different dataset prevents direct comparison, and GAN-plus-deep-learning training is costly for real-time use.

F. Comparative study

Tables I and II compare the works on approach, evaluation and auditing.

G. Research gaps addressed

- 1) **Unaudited preprocessing.** No prior work tests whether CICEVSE2024 accuracies are artefacts of capture-session leakage.
- 2) **Single-seed evaluation.** With classes of 28 to 81 flows, single-seed results are unreliable; we report mean and standard deviation over three seeds.
- 3) **Accuracy as the headline.** Accuracy is dominated by flood classes; macro- F_1 exposes the DoS-versus-reconnaissance gap.
- 4) **No per-attack or operational view.** No prior work reports which model to trust for which attack, false-alarm rates, or what an operator should do.
- 5) **Validity of the drift claim.** We test whether detected drift reflects traffic evolution or dataset assembly.

III. PROBLEM FORMULATION

A. Threat model and placement

The adversary is any device on the charging site’s network, or a compromised vehicle on the charging cable, that sends reconnaissance or flooding traffic to a charging station. The detector is *passive*: it receives a copy of the traffic from a mirrored switch port, classifies each completed flow and raises an alert. It does not block traffic; mitigation is left to people and policy, supported by the response guidance of Section VI.

B. Classification task

Each network flow is summarised by a feature vector $\mathbf{x} \in \mathbb{R}^d$ ($d = 60$ after correction) with a label $y \in \mathcal{Y}$, $|\mathcal{Y}| = K = 15$. A classifier $f : \mathbb{R}^d \rightarrow \mathcal{Y}$ is learned from training flows. For operational use we also map each class to a response category,

$$c : \mathcal{Y} \rightarrow \{\text{DoS}, \text{Recon}, \text{Benign}\}, \quad (1)$$

which groups the eight denial-of-service classes, the six reconnaissance classes and benign traffic.

TABLE I
COMPARATIVE STUDY OF REFERENCE WORKS AND EVNET SENTINEL

Study	Type	Dataset	Best model	Reported	Online	Drift	Leakage audit
Bean et al. [1]	Survey	CICEVSE2024 (review)	—	—	cited	cited	×
Joglekar et al. [2]	Hybrid IDS	CICEVSE2024	XGBoost	99.99% acc.	×	×	IP/MAC only
Malimage et al. [3]	Threat analysis	—	—	—	—	—	—
Makhmudov et al. [4]	Online IDS	CICEVSE2024	ARF+ADWIN	98.40% acc.	✓	✓	×
Tanyildiz et al. [5]	Pre-warning	EVS One	GAN-MLP	$R^2 = 0.74$	×	×	—
EVNet Sentinel	Audited IDS	CICEVSE2024	RF (corrected)	0.558 macro-F_1	✓	✓	✓ (5 findings)

TABLE II
EXTENDED COMPARISON: METHODOLOGY AND FINDINGS

Study	Detection scope	Evaluation	Deployment	Key strength	Key limitation
Bean et al. [1]	Network, host, power (reviewed)	—	—	Taxonomy; challenges	open
Joglekar et al. [2]	Network + host	Single 80/10/10	seed, Offline	First EVCS hybrid IDS	No drift handling; timestamps kept
Malimage et al. [3]	Industry threat landscape	—	—	Real breaches; regulation	No detector
Makhmudov et al. [4]	Network	Single prequential	seed, Simulated stream	First online EVCS IDS	Leakage; autocorrelation; drift on seams
Tanyildiz et al. [5]	Time to next attack	Train/test split	Offline	Predictive pre-warning	Other dataset; heavy compute
EVNet Sentinel	Network, passive (mirror port)	3 seeds, macro-F_1, per attack	API + web front end	Leakage audit; realistic ceiling; per-attack guidance	Network only; one testbed; few benign flows

C. Metrics

For class k with precision P_k and recall R_k , the per-class score and the headline metric are

$$F_{1,k} = \frac{2P_k R_k}{P_k + R_k}, \quad \text{macro-}F_1 = \frac{1}{K} \sum_{k=1}^K F_{1,k}. \quad (2)$$

Macro- F_1 weights every class equally, whereas accuracy and weighted F_1 weight classes by their support n_k and are therefore dominated by the four large flood classes. Every result is reported as the mean $\bar{m}$ and sample standard deviation s over $S = 3$ seeds,

$$\bar{m} = \frac{1}{S} \sum_{s=1}^S m_s, \quad s = \sqrt{\frac{1}{S-1} \sum_{s=1}^S (m_s - \bar{m})^2}, \quad (3)$$

where each seed changes both the data split and the model initialisation.

IV. DATASET AND TESTBED

A. CICEVSE2024

The CICEVSE2024 dataset of the Canadian Institute for Cybersecurity [8] provides network traffic from a testbed with two charging stations, EVSE-A and EVSE-B, recorded while idle or charging (Table III). Flows were extracted with NFStream into 86 features and are labelled from the capture file name.

TABLE III
DATASET PROVENANCE

Property	Value
Source	CIC, University of New Brunswick
Capture files	59 (2 stations; idle, charging)
Raw flows	2,744,700
Classes	15 (14 attack + benign)
Feature extractor	NFStream, 86 features

The 15 classes are five volumetric floods (SYN, TCP, UDP, PSH-ACK, SynonymousIP), six nmap reconnaissance modes (TCP port scan, SYN stealth scan, OS fingerprinting, service-version detection, vulnerability scan, aggressive scan), three low-rate or sparse DoS classes (ICMP flood, ICMP fragmentation, Slowloris) and benign traffic.

B. Who attacks what

The dataset documents its devices and their MAC addresses but not which device attacked which. Since every flow records its source and destination MAC address, we derived the route of each of the 57 attack captures (Table IV). Outside the tiny ICMP captures, 99–100% of the flows in each capture follow the listed route.

Two consequences follow. First, *every attack targets a charging station*; none targets the CSMS or the vehicle, so the CSMS is affected only indirectly, when a flooded station can no longer keep its OCPP session. Second, in six captures the

TABLE IV
ATTACK ROUTES, FROM THE MAC ADDRESSES IN THE 57 ATTACK CAPTURES

Attacker	Target	Caps.	Attacks
Kali PC (wifi)	EVSE-A	24	all but UDP flood
Kali PC (wifi)	EVSE-B	23	all but UDP flood, Slowloris
Raspberry Pi (wifi)	both	4	UDP flood
Compromised EV (V2G)	EVSE-B	6	the six scans

TABLE V
PREPROCESSING STAGE COUNTS

Stage	Rows	Columns
Raw concatenation	2,744,700	86 + label
After column drops	2,744,700	65
After global deduplication	1,198,151	65
Feature matrix	1,198,151	60
Train / validation / test	838,705 / 179,723 / 179,723	60

vehicle is the attacker, scanning EVSE-B over the ISO 15118 link. The testbed topology shipped with the dataset shows that traffic was captured from a mirrored switch port, which fixes the deployment model of Section III.

C. Corrected preprocessing

Our pipeline (`src/data_prep/preprocess.py`) proceeds as follows.

- 1) **Concatenation and labelling.** All 59 captures are concatenated and labelled from file names through a 19-alias map.
- 2) **Identifier drops.** Flow IDs, IP, MAC and vendor identifiers, and sparse application-layer string fields are removed as environment-specific.
- 3) **Leakage removal.** The six absolute timestamp columns (`*_first_seen_ms`, `*_last_seen_ms`) and `src_port` are removed (Section V); `dst_port` is kept.
- 4) **Constant columns** (`vlan_id`, `tunnel_id`) are removed.
- 5) **Deduplication** on the reduced feature set.
- 6) **Standardisation** of each feature j with training mean μ_j and deviation σ_j : $z_j = (x_j - \mu_j)/\sigma_j$.
- 7) **Stratified split** 70/15/15.

Table V traces the row and column counts.

V. LEAKAGE ANALYSIS

A. Finding 1: the label is recoverable from one timestamp

Each attack was recorded in its own capture file at its own wall-clock time. An absolute timestamp therefore identifies the *recording*, and through it the label, without describing the traffic. We test this with a single-feature probe: a decision tree f_j of depth 25 trained on feature j alone, scored by 3-fold cross-validated accuracy,

$$\text{Acc}(f_j) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[f_j(x_{ij}) = y_i]. \quad (4)$$

TABLE VI
SINGLE-FEATURE LABEL RECOVERY (15 CLASSES, 3-FOLD CV)

Feature	Nature	Accuracy
<code>bidirectional_first_seen_ms</code>	fingerprint	0.9976
<code>src_port</code>	partial	0.4028
<code>dst_port</code>	legitimate	0.2917
<i>Published multiclass accuracy [4]</i>		<i>0.9840</i>

TABLE VII
FINGERPRINT ABLATION: FLOWS REMAINING AFTER DEDUPLICATION

Timestamps	<code>dst_port</code>	Total	Recon.	%
kept	kept	2,728,799	1,724,504	100.0
kept	dropped	1,277,520	273,932	15.9
dropped	kept	1,198,143	194,737	11.3
dropped	dropped	977,862	6,821	0.4

The timestamp alone scores 0.9976, above the published 0.9840 for the full model (Table VI). The reference preprocessing [4] keeps all six timestamp columns, and in our own pre-correction Random Forest 39.8% of the total feature importance sat on them.

B. Finding 2: the legitimate feature is the one discarded

Reconnaissance flows are distinguished almost entirely by `dst_port` and by the timestamp. Table VII crosses keeping or dropping each before deduplication. The published configuration (bold) discards `dst_port` — the feature that defines a port scan, which sweeps destination ports — yet keeps the timestamps, so its 273,932 reconnaissance flows are separable by *when* they were captured rather than by what they did.

C. Finding 3: stream order inflates prequential accuracy

A streaming learner is evaluated *prequentially*: each instance is first predicted, then used for training. After t instances,

$$\text{Acc}_t = \frac{1}{t} \sum_{i=1}^t \mathbb{I}[f_{i-1}(\mathbf{x}_i) = y_i], \quad (5)$$

where f_{i-1} has seen only the first $i - 1$ instances. This is sensitive to the order of the stream. Define the label autocorrelation

$$\rho = \Pr[y_t = y_{t-1}], \quad (6)$$

which is exactly the accuracy of the trivial rule “predict the previous label”. In capture order each file holds one class, so labels change only at file boundaries: in the 1,921,182-flow training stream the label changes 54 times, giving $\rho = 0.99997$. For a uniformly shuffled stream with class proportions p_k ,

$$\rho_{\text{shuffled}} = \sum_{k=1}^K p_k^2 = 0.117. \quad (7)$$

Table VIII shows that in capture order ARF+ADWIN (0.9997) does not even beat the repeat-last rule (0.99997): its accuracy measures label autocorrelation, not learning. On

TABLE VIII
EFFECT OF STREAM ORDER ON PREQUENTIAL ACCURACY (ARF+ADWIN,
LEAK-FREE, 1.92M FLOWS)

Order	ARF acc.	Repeat-last ρ	Drift events
Capture order	0.9997	0.99997	43
Shuffled	0.5585	0.117	14

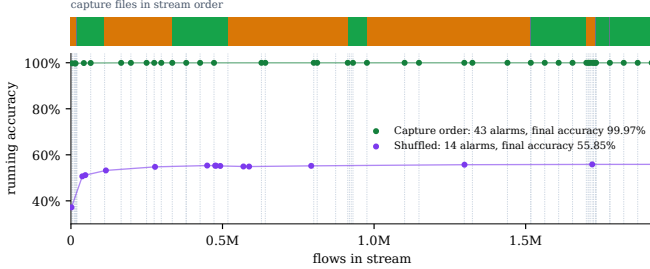


Fig. 1. ADWIN alarms over the training stream. Top band: the 58 capture files in stream order (green volumetric, orange reconnaissance, grey low-rate, blue benign); dashed lines mark file seams.

the shuffled stream it scores 0.5585, well above both the repeat-last rule and the majority class (0.193); this is the meaningful figure, and it is 44 points lower.

D. Finding 4: the drift detector finds file seams

ADWIN [7] keeps a window W of recent error indicators and declares drift when some split $W = W_0 W_1$ has sub-window means that differ by more than

$$\epsilon_{\text{cut}} = \sqrt{\frac{1}{2m} \ln \frac{4}{\delta'}}, \quad m = \frac{1}{1/n_0 + 1/n_1}, \quad \delta' = \frac{\delta}{n}, \quad (8)$$

where n_0, n_1 are the sub-window sizes and $n = |W|$. In capture order, each file seam is an abrupt distribution change by construction. Let $b_1, \dots, b_B$ be the seam positions and $e_1, \dots, e_E$ the drift alarms. The distance of each alarm to its nearest seam and the number of aligned alarms are

$$d_i = \min_j |e_i - b_j|, \quad H = \sum_{i=1}^E \mathbb{I}[d_i \leq w], \quad (9)$$

with $w = 100$ flows. Under a null model that places the E alarms uniformly at random, $R = 2000$ draws give $H^{(r)}$ and the empirical significance

$$p = \frac{1 + \sum_{r=1}^R \mathbb{I}[H^{(r)} \geq H]}{1 + R}. \quad (10)$$

In capture order $H = 42$ of $E = 43$ alarms (median distance 22 flows), against 2.2 ± 1.5 under the null, so $p < 0.0005$ (Fig. 1). The detector is locating the seams of dataset assembly, not drift in charging traffic — the property offered in [4] as the method’s advantage.

TABLE IX
EFFECT OF REMOVING `SRC_PORT` (MACRO- F_1)

Model	With	Without	Δ
Random Forest	0.5894	0.5138	-0.0756
Decision Tree	0.6834	0.5135	-0.1699

E. Finding 5: `src_port` is a residual fingerprint

Operating systems allocate ephemeral source ports near-sequentially, so the ports of one capture occupy a contiguous band. With rows and split held fixed and only `src_port` removed (Table IX), the deeper Decision Tree loses more than twice as much as Random Forest, the signature of a memorisable fingerprint: extra depth buys finer port bands and nothing else. Without it the two converge, so the tree’s earlier advantage was memorisation.

VI. SYSTEM ARCHITECTURE

Fig. 2 summarises the system from capture to response.

A. Placement

EVNet Sentinel sits where CICEVSE2024 was recorded: on the mirrored port of the switch that carries station, CSMS and vehicle traffic. It therefore sees every station’s traffic, including ISO 15118 traffic from a vehicle, without being in its path or modifying station firmware.

B. Models and prediction service

Two pipelines are trained offline on the corrected features: static scikit-learn classifiers (Random Forest, Decision Tree, and SGD-trained Logistic Regression and linear SVM), and the River [9] implementation of ARF+ADWIN with the configuration of [4]. A FastAPI service loads every model and the matching scaler into a registry at start-up, tolerating individual load failures, and classifies one flow per request; the caller chooses the model by name.

C. Response guidance

Each detection maps to actions for three roles — the station owner, the CSMS operator and the network analyst — with a severity per attack. The guidance uses the per-attack reliability of Section VIII-D: when the active model does not name the predicted attack reliably, it reports and acts on the category $c(\hat{y})$ of (1) instead, with the severity range of that category, because acting on a confidently wrong name is worse than acting on a correct category.

D. Front end

A Next.js application is built entirely from exported results (no live inference), which keeps every displayed number reproducible. It provides an interactive landing page on which visitors aim an attack at a station and see the verdict the model actually returned for a real held-out flow, and can move the naming-confidence bar of Section VIII-E; a simulation console that replays held-out flows from the device that really sent them to the station they really hit (Table IV), with scripted

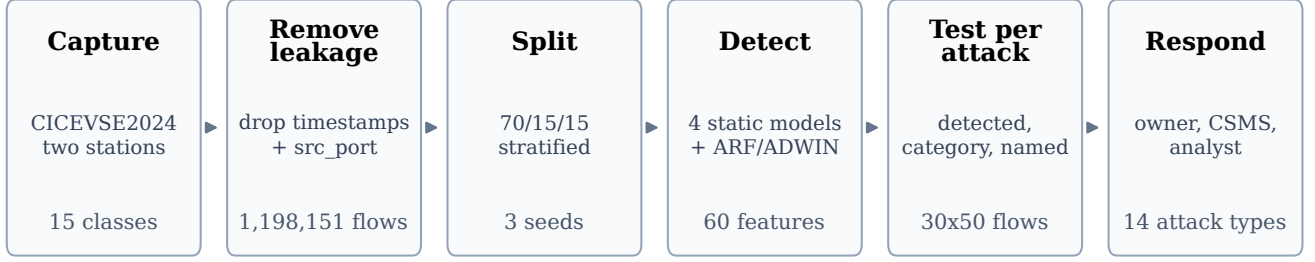


Fig. 2. Method and system at a glance, from capture to response.

multi-attack campaigns and an incident log; and a dashboard with the findings, per-attack results and figures of this paper.

VII. EXPERIMENTAL METHODOLOGY

A. Models

Random Forest uses 100 trees of maximum depth 15; Decision Tree a maximum depth of 20; Logistic Regression and the linear SVM are trained by stochastic gradient descent with log and hinge loss. All four use balanced class weights,

$$w_k = \frac{N}{K n_k}, \quad (11)$$

so that a misclassified flow of a rare class costs as much in total as one of a common class. ARF [6] uses 20 trees, `max_features` = 0.5, a grace period of 30 and ADWIN with $\delta = 0.002$, as in [4].

B. Evaluation protocol

Every headline number follows (3) over seeds 42, 1337 and 2024. With Benign at 81 flows and ICMP fragmentation at 28, the split is the dominant source of run-to-run variation, which a single seed hides. Macro- F_1 (2) is the headline: on identical predictions the corrected Random Forest scores 0.868 accuracy but 0.558 macro- F_1 .

C. Per-attack test harness

A single score per model hides which attacks it handles. For each attack class a , model f and seed, we draw $T = 30$ batches $\mathcal{B}$ of $n = 50$ held-out flows of class a and score each batch at three levels:

$$D = \frac{1}{n} \sum_{i \in \mathcal{B}} \mathbb{I}[\hat{y}_i \neq \text{Benign}],$$

$$C = \frac{1}{n} \sum_{i \in \mathcal{B}} \mathbb{I}[c(\hat{y}_i) = c(a)], \quad E = \frac{1}{n} \sum_{i \in \mathcal{B}} \mathbb{I}[\hat{y}_i = a], \quad (12)$$

the *detected*, *category* and *exact* rates, with $E \leq C \leq D$. Each rate is reported as its mean over trials with a 95% interval $\bar{r} \pm 1.96 s_r / \sqrt{T}$. A model's verdict for an attack follows a ladder with reliability threshold $\tau = 0.9$: *identified* if $E \geq \tau$,

else *category* if $C \geq \tau$, else *detected* if $D \geq \tau$, else *missed*. Benign traffic is scored by its false-alarm rate

$$\text{FA} = \frac{1}{|\mathcal{B}_0|} \sum_{i \in \mathcal{B}_0} \mathbb{I}[\hat{y}_i \neq \text{Benign}], \quad (13)$$

over the benign flows $\mathcal{B}_0$. Mixed campaigns send four attacks at once (25 flows each, 30 trials). The harness fails if a regression gate is crossed: each flood named exactly at ≥ 0.95 , each attack detected at ≥ 0.95 , and each scan categorised at ≥ 0.85 by at least one model.

D. Per-feature separability

To ask whether a ceiling lies in the data or in the models, we measure, inside one category g (DoS or reconnaissance), how much of the uncertainty about the class each feature X_j removes on its own:

$$S_j^{(g)} = \frac{I(X_j; Y | c(Y) = g)}{H(Y | c(Y) = g)}, \quad H(Y) = - \sum_k p_k \log p_k, \quad (14)$$

where I is estimated with a k -nearest-neighbour mutual-information estimator on 6,000 held-out flows per category. $S_j = 1$ means feature j alone determines the class; $S_j = 0$ means it carries no information about it. Unlike permutation importance, (14) does not depend on any model.

E. Confidence-gated naming

Given class probabilities $\hat{p}(k | \mathbf{x})$, the model names the attack only when its top probability reaches a bar t , and otherwise reports only the category:

$$\text{output}(\mathbf{x}) = \begin{cases} \arg \max_k \hat{p}(k | \mathbf{x}) & \text{if } \max_k \hat{p}(k | \mathbf{x}) \geq t, \\ c(\arg \max_k \hat{p}(k | \mathbf{x})) & \text{otherwise.} \end{cases} \quad (15)$$

Over attack flows this splits outcomes into the shares named correctly $R(t)$, named wrongly $W(t)$, and reported by category only $H(t) = 1 - R(t) - W(t)$. Raising t trades naming coverage for correctness.

F. Choice of primary detector

Counting per-attack wins would reward a model for being least wrong on scans it rarely names. We instead take the models whose mean exact rate $\bar{E}_f$ over well-sampled attacks

TABLE X
MULTICLASS RESULTS, MEAN $\pm$ STD OVER 3 SEEDS (CORRECTED FEATURES)

Model	Macro- F_1	W- F_1	Acc.	Fit (s)
Random Forest	0.558$\pm$0.008	0.865	0.868	8.5
Decision Tree	0.557 $\pm$ 0.010	0.868	0.868	3.4
Log. Regression	0.436 $\pm$ 0.017	0.853	0.863	78.9
SVM	0.411 $\pm$ 0.017	0.849	0.862	34.9

TABLE XI
MACRO- F_1 BEFORE AND AFTER REMOVING THE TIMESTAMPS

Model	Leaked	Corrected	Δ
Random Forest	0.9547	0.5894	-0.3653
Log. Regression	0.4622	0.3308	-0.1314
SVM	0.2829	0.3084	+0.0255

TABLE XII
PER-CLASS F_1 , RANDOM FOREST, MEAN OVER 3 SEEDS

Class	Group	Support	F_1
PSH-ACK flood	DoS	195,952	1.000
SynonymousIP flood	DoS	256,739	1.000
TCP flood	DoS	256,297	1.000
SYN flood	DoS	259,484	1.000
UDP flood	DoS	32,072	0.999
TCP port scan	Recon.	35,123	0.265
OS fingerprinting	Recon.	27,702	0.223
Vulnerability scan	Recon.	39,002	0.204
Service-version detection	Recon.	30,338	0.156
Aggressive scan	Recon.	27,815	0.145
SYN stealth scan	Recon.	34,765	0.105
Benign	Other	81	0.986
ICMP flood	Other	32	0.660
ICMP fragmentation	Other	28	0.439
Slowloris	Other	2,721	0.191

is within 0.02 of the best, and among them choose the one with the fewest false alarms:

$$f^* = \arg \min_{f: \bar{E}_f \geq \max_g \bar{E}_g - 0.02} \text{FA}_f. \quad (16)$$

VIII. RESULTS AND DISCUSSION

A. Corrected benchmarks

Random Forest and Decision Tree are indistinguishable within one standard deviation (Table X), which independently confirms Finding 5. The linear models trail by more than 0.1 macro- F_1 . All four reach about 0.86 accuracy, illustrating why accuracy alone cannot separate them.

On a like-for-like split, removing the timestamps costs Random Forest 0.365 macro- F_1 , whereas the linear models barely move (Table XI). A tree can carve a timestamp axis into one interval per capture file; a hyperplane cannot. The gap between tree and linear models in the reference work was therefore largely a measure of how well each could exploit the leak.

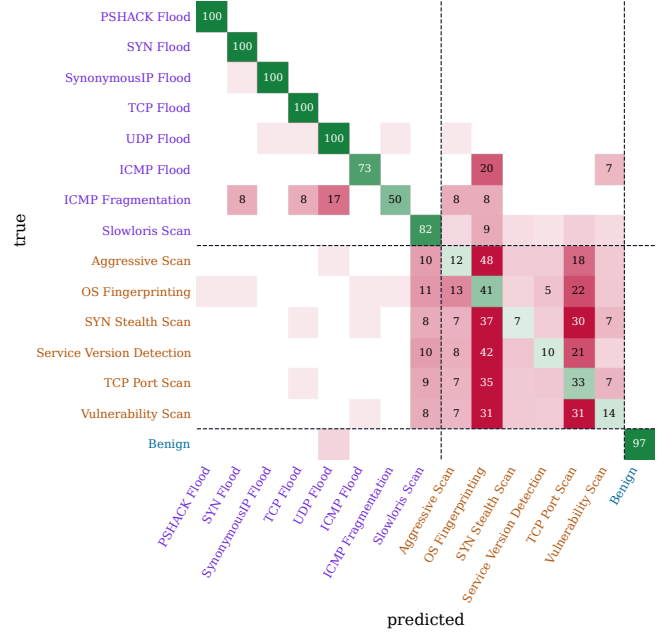


Fig. 3. Random Forest confusion matrix, row-normalised and averaged over three seeds (%; cells under 5% unlabelled). Dashed lines separate DoS, reconnaissance and benign.

B. Per-class results: the reconnaissance ceiling

The dichotomy is stark (Table XII): volumetric floods reach $F_1 = 1.000$ with zero variance across seeds, whereas reconnaissance averages 0.183. Sample size does not explain it: UDP flood (32,072 flows) scores 0.999 while TCP port scan (35,123 flows) scores 0.265. All six scans are nmap modes, and nmap -A is by definition -sV -O -sC --traceroute, a superset of two other modes.

C. Why the scans cannot be separated

The errors are structured (Fig. 3). First, *scans collapse into one scan*: Random Forest labels 32–48% of every other scan as OS fingerprinting, while Logistic Regression absorbs them into TCP port scan instead. Which class becomes the sink depends on the model; that a sink exists does not. Second, *one confusion crosses the category boundary*: 8–11% of every scan is labelled Slowloris by Random Forest (3% by Logistic Regression), because both hold long-lived, near-idle connections. This is almost all of the error between reconnaissance and DoS.

Equation (14) tests whether this is a limit of the models or of the data (Fig. 4). Inside DoS a single inter-arrival-time feature, `src2dst_mean_piat_ms`, resolves $S_j = 0.98$ of the class uncertainty. Inside reconnaissance the best feature resolves 0.05. No flow statistic distinguishes the six nmap modes, so no classifier trained on these features can, whatever its capacity. Permutation importance, tried first, reads near zero for every scan feature because the features are redundant — shuffling one leaves the others to carry the same information — which is why a model-free measure is needed.

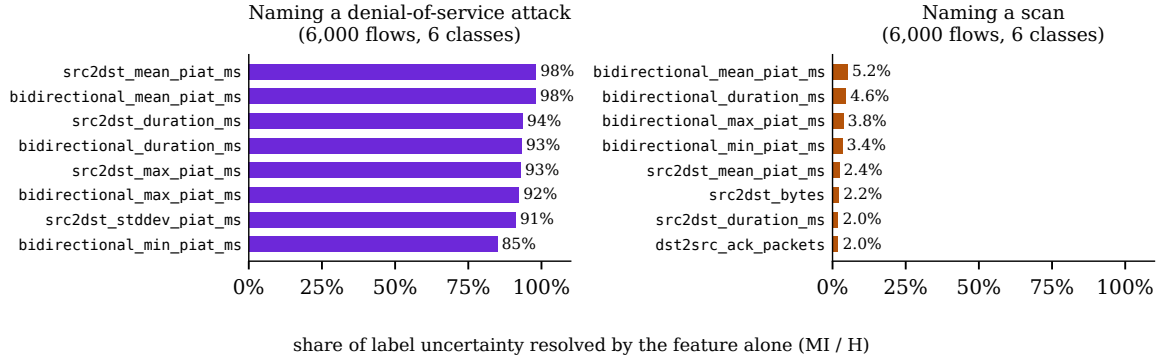


Fig. 4. Per-feature separability S_j of (14) inside DoS and inside reconnaissance flows (6,000 held-out flows each). Both panels share one axis.

TABLE XIII
PER-ATTACK EXACT-NAME RATE E FOR RANDOM FOREST, AND THE BEST MODEL AT NAMING (E) AND AT CATEGORISING (C)

Attack	RF	Best E	C	Best C
5 volumetric floods	1.00	all (≥ 0.98)	1.00	all
Slowloris	0.82	RF	0.82	RF
ICMP flood ^a	0.74	RF	0.74	RF
ICMP fragmentation ^a	0.49	LR 0.58	0.82	SVM
TCP port scan	0.32	LR 0.65	0.97	LR
SYN stealth scan	0.06	SVM 0.43	0.97	LR
OS fingerprinting	0.41	RF/DT 0.41	0.95	LR
Vulnerability scan	0.13	SVM 0.26	0.97	LR
Aggressive scan	0.12	DT 0.15	0.94	LR
Service-version det.	0.10	DT 0.13	0.96	LR

^aFewer than 100 held-out flows; batches resampled.

TABLE XIV
FALSE ALARMS ON BENIGN TRAFFIC, BY MODEL

	RF	DT	LR	SVM
False-alarm rate (13)	0.02	0.12	0.46	0.59

D. Per-attack detection and model choice

Three results follow (Tables XIII and XIV). First, *nothing goes unnoticed*: across 90 mixed campaigns every model flags at least 99.9% of attack flows as malicious ($D \geq 0.999$), so the system’s weakness is naming, not noticing. Second, floods are named exactly by every model, while scans are recognised as reconnaissance 96% of the time by the best model for each (C) but named only 34% (E), with no single model best at naming them. Third, false alarms separate the models sharply. Applying (16), Random Forest and Decision Tree name attacks equally well, and Random Forest raises far fewer false alarms, so **Random Forest is the primary detector**, with Logistic Regression a useful second opinion on whether traffic is a scan, where it is best for all six. The false-alarm rates rest on 36 benign flows and are indicative only.

E. How sure before naming?

Table XV quantifies the response rule of Section VI. Without a bar ($t = 0$) Random Forest names 81% of scans wrongly.

TABLE XV
CONFIDENCE-GATED NAMING (15): SHARES NAMED RIGHT R , WRONG W , CATEGORY ONLY H

Model	t	DoS			Reconnaissance		
		R	W	H	R	W	H
RF	0	0.97	0.03	0.00	0.19	0.81	0.00
RF	0.5	0.93	0.00	0.07	0.03	0.03	0.94
RF	0.9	0.87	0.00	0.13	0.01	0.00	0.99
DT	0	0.95	0.05	0.00	0.20	0.81	0.00
DT	0.5	0.92	0.00	0.07	0.04	0.06	0.91
DT	0.9	0.88	0.00	0.12	0.03	0.01	0.96

At $t = 0.5$ wrong names fall to 3% and 94% of scans are reported as reconnaissance — a correct and actionable alert — while DoS keeps 93% exact naming. The single Decision Tree expresses doubt less well: at $t = 0.9$ it still names 1% of scans wrongly where Random Forest names none, because a leaf’s class frequencies are a coarser confidence signal than an average over 100 trees. Logistic Regression is excluded: its multiclass probabilities saturate, so their arg-max disagrees with its own predictions.

IX. REPRODUCIBILITY

All results regenerate from the public repository¹ with pinned dependencies; River’s API changed materially around version 0.21, and scikit-learn’s tree splitting has shifted across minor releases. The key scripts are:

- preprocess.py: corrected pipeline;
- run_experiments.py: multi-seed results and models;
- replay_reference_preprocessing.py, ablate_fingerprints.py, drift_boundary_alignment.py, srcport_control.py: Findings 1–5;
- attack_test_harness.py: per-attack tests and regression gates; attack_routes.py: Table IV;
- make_paper_figures.py: every figure, drawn from the same exported data the front end displays.

¹https://github.com/Mukta01/EVNet_Sentinel

X. LIMITATIONS AND THREATS TO VALIDITY

- 1) **Label noise.** Labels come from capture file names, so background traffic recorded during an attack carries the attack label; part of the reconnaissance confusion may be such flows, which no detector could name correctly.
- 2) **Benign sparsity.** Only 81 benign flows exist. Binary attack-versus-benign classification is answered by always predicting “attack” and is excluded, and false-alarm rates are indicative.
- 3) **Random split.** Headline results use a stratified random split for comparability; deduplication removes exact repeats but not near-duplicates from one capture on both sides. Grouped and temporal splits are implemented but not reported here.
- 4) **One testbed.** Two stations, one CSMS and standard tools at default settings; a paced or disguised scan changes exactly the timing features the detector relies on.
- 5) **Untested responses and unused modalities.** The response guidance is standard practice and was not itself measured; CICEVSE2024’s host-event and power traces, which may separate the scans, are not used.

XI. CONCLUSION AND FUTURE WORK

The high accuracies reported for ML-based intrusion detection on CICEVSE2024 are largely an artefact of how the dataset was assembled: one capture timestamp beats the published model, streaming accuracy in capture order is matched by repeating the previous label, and the drift detector fires on the seams between capture files. With these effects removed, volumetric floods remain trivially separable while the six nmap scan modes are not, and per-feature mutual information shows that this ceiling lies in the flow statistics, not in model capacity. Operationally, no attack goes unnoticed; Random Forest names attacks as well as any model while raising the fewest false alarms; and requiring confidence before naming turns most wrong scan names into correct category alerts.

Future work should (a) model the packet sequence within a flow, which flow aggregation discards and which may separate the scan modes; (b) fuse host-event and power data; and (c) evaluate sequence models on grouped splits against the repeat-last baseline of (6), which any model on an ordered stream must beat to show learning.

ACKNOWLEDGMENT

We build on the open-source implementation of [4] and thank the Canadian Institute for Cybersecurity for the CICEVSE2024 dataset. This work was carried out for the Software Engineering course at Vidyalankar Institute of Technology.

REFERENCES

- [1] J. Bean, V. Manias et al., “Cybersecurity of electric vehicle charging infrastructure: Recent advances, open challenges, and future directions,” arXiv:2605.24190v1, May 2026.
- [2] C. Joglekar et al., “A hybrid intrusion detection system for electric vehicle charging infrastructure,” arXiv:2606.23236v1, 2026.
- [3] K. Malimage et al., “Cybersecurity challenges in the electric vehicle market,” in *Proc. CIGRE Canada Conference*, 2023, pp. 1–6.
- [4] F. Makhmudov, D. Kilichev, U. Giyosov, and F. Akhmedov, “Online machine learning for intrusion detection in electric vehicle charging systems,” *Mathematics*, vol. 13, no. 5, p. 712, 2025, DOI: 10.3390/math13050712.
- [5] E. Tanyildiz et al., “Detection of cyber attacks in electric vehicle charging systems using a remaining useful life generative adversarial network,” *Scientific Reports*, vol. 15, 2025.
- [6] H. M. Gomes, A. Bifet, J. Read, J. P. Barddal, F. Enembreck, B. Pfharinger, G. Holmes, and T. Abdessalem, “Adaptive random forests for evolving data stream classification,” *Machine Learning*, vol. 106, no. 9–10, pp. 1469–1495, 2017.
- [7] A. Bifet and R. Gavaldà, “Learning from time-changing data with adaptive windowing,” in *Proc. SIAM Int. Conf. Data Mining (SDM)*, 2007, pp. 443–448.
- [8] Canadian Institute for Cybersecurity, “CICEVSE2024 dataset,” University of New Brunswick, 2024. [Online]. Available: <https://www.unb.ca/cic/datasets/>
- [9] J. Montiel, M. Halford, S. M. Mastelini et al., “River: Machine learning for streaming data in Python,” *J. Mach. Learn. Res.*, vol. 22, no. 110, pp. 1–8, 2021.